



**INSTITUTO SUPERIOR POLITÉCNICO
DEPARTAMENTO DE ENGENHARIA
INFORMÁTICA**

TRABALHO EM GRUPO DE POO II

Tema: Sistema de Gestão de Companhia Aérea

Integrantes do grupo:

- 1- Rosa Sozinho Garcia
- 2- Henriques Calenda Cassanda
- 3- Luyeye Pedro

Turma: 101

Ano: 3º

Período: Manhã

ORIENTADO POR

Joaquim João Nsaku Ventura

INTRODUÇÃO

O presente relatório descreve o desenvolvimento do projecto denominado **Sistema de Gestão de Companhia Aérea**, implementado em linguagem C# utilizando o paradigma de Programação Orientada a Objectos (POO) numa aplicação Console Application.

O sistema foi desenvolvido com o objectivo de automatizar os processos de operação de uma companhia aérea regional, incluindo o cadastro de aeronaves e passageiros, a criação e gestão de voos, a efectuação e cancelamento de reservas, a emissão de bilhetes de passagem, o processo de check-in e a confirmação de embarque.

A implementação foi baseada nos princípios fundamentais da Programação Orientada a Objectos, incluindo encapsulamento, composição, associação, utilização de enums, colecções genéricas (List<T> e Dictionary<TKey, TValue>), LINQ e tratamento de excepções personalizadas.

OBJECTIVOS

Objectivo Geral

Desenvolver um sistema computacional capaz de gerir aeronaves, voos, passageiros, reservas, bilhetes e o processo de check-in de uma companhia aérea regional.

Objectivos Específicos

- Implementar um sistema orientado a objectos em C#
- Cadastrar aeronaves com capacidade por classe (Económica, Executiva, 1.^a Classe)
- Criar e gerir voos com rota, horário, aeronave associada e preços dinâmicos
- Efectuar reservas com selecção de classe e atribuição automática de assento
- Calcular preços com base na antecedência da compra
- Cancelar reservas com libertação automática do assento
- Realizar check-in dentro da janela permitida (24h a 1h antes da partida)
- Emitir bilhetes de passagem com todos os dados do voo e passageiro
- Aplicar conceitos avançados de POO: composição, associação, enums e LINQ
- Implementar excepções personalizadas para situações de erro clínicas
- Desenvolver um sistema funcional em Console Application

CONTEXTUALIZAÇÃO DO PROBLEMA

As companhias aéreas regionais necessitam de sistemas informáticos para organizar as suas operações, controlar a disponibilidade de assentos e gerir o fluxo de passageiros de forma eficiente.

Sem um sistema automatizado, o controlo manual pode provocar:

- Overbooking por falta de controlo em tempo real da ocupação
- Atribuição incorrecta ou duplicada de assentos
- Emissão de bilhetes sem registo histórico adequado
- Impossibilidade de aplicar políticas de preço dinâmico
- Falhas no controlo do processo de check-in e embarque
- Dificuldades na geração de relatórios de receita e ocupação

O sistema desenvolvido surge como solução para melhorar a organização e eficiência operacional da companhia aérea AirPortugal Regional.

METODOLOGIA UTILIZADA

Para o desenvolvimento do projecto foram utilizadas as seguintes etapas:

1. Levantamento dos requisitos operacionais e funcionais da companhia aérea
2. Identificação das classes principais e das suas relações
3. Criação do diagrama UML de classes
4. Implementação em C# com .NET Framework 4.6 (Visual Studio 2015)
5. Aplicação dos conceitos de POO: encapsulamento, composição, associação
6. Testes, validação e correcção de erros
7. Documentação do projecto

FUNDAMENTAÇÃO TEÓRICA

Programação Orientada a Objectos

A Programação Orientada a Objectos é um paradigma de programação baseado na criação de classes e objectos. Os principais conceitos utilizados neste projecto foram:

Encapsulamento

Todos os atributos das classes foram encapsulados com propriedades públicas de leitura (**get**) e escrita privada (**private set**), garantindo a integridade dos dados e evitando modificações externas indevidas. Exemplo: na classe **Passageiro**, propriedades como **Nome**, **NumeroPassaporte** e **Email** são definidas com **private set**.

Composição

Relação forte entre objectos em que um objecto só existe no contexto do outro. A classe **Reserva** possui composição com a classe **Bilhete**: o Bilhete é criado automaticamente no construtor da Reserva e não existe de forma independente — se a Reserva for cancelada, o Bilhete é automaticamente invalidado.

Associação

Relação entre classes independentes. A classe **Voo** associa-se a **Aeronave**, **Passageiro** e à lista de **Reservas**. O **GestorCompanhia** agrega listas de Aeronaves, Voos, Passageiros e Reservas de forma independente.

Enums

ClasseVoo (Económica, Executiva, PrimeiraClasse) permite classificar os assentos e calcular preços por segmento. **EstadoReserva** (Activa, Cancelada, CheckInRealizado, Embarcado) controla o ciclo de vida completo de cada reserva.

Tratamento de Excepções

Foram implementadas três excepções personalizadas: **AssentoOcupadoException** (assento já reservado), **VooLotadoException** (voo sem lugares disponíveis na classe pretendida) e **CheckInForaDoPrazoException** (check-in fora da janela de 24h a 1h).

ANÁLISE DE REQUISITOS

Requisitos Funcionais

O sistema deve permitir:

- Cadastro de aeronaves com matrícula, tipo e capacidade por classe
- Criação de voos com rota (origem/destino IATA), horário e preços por classe
- Cadastro de passageiros com nome, passaporte, email, telefone e data de nascimento
- Efectuação de reservas com selecção de classe e atribuição automática de assento
- Cálculo dinâmico do preço em função da antecedência da compra
- Cancelamento de reservas com libertação do assento e invalidação do bilhete
- Realização de check-in dentro da janela permitida (24h a 1h antes da partida)
- Confirmação de embarque após check-in realizado
- Emissão gráfica de bilhetes de passagem com todos os dados
- Listagem de passageiros por voo
- Emissão de relatório geral com receita total e estado das reservas

Requisitos Não Funcionais

- Interface interactiva em consola com menus numerados e sub-menus
- Código organizado em namespaces e ficheiros separados por responsabilidade
- Fácil manutenção e extensão do sistema
- Tratamento robusto de erros com mensagens claras ao utilizador
- Compatibilidade com Visual Studio 2015 e .NET Framework 4.6

MODELAÇÃO UML

O sistema foi modelado utilizando Diagrama de Classes UML, reflectindo a estrutura e as relações entre os componentes implementados.

Principais Classes

- **Passageiro** — dados pessoais, passaporte, cálculo de idade
- **Aeronave** — matrícula, tipo, lista de assentos por classe, percentagem de ocupação
- **Assento** — número, classe e estado de ocupação (livre/ocupado)
- **Voo** — número, rota, horário, aeronave, reservas e preços por classe
- **Bilhete** — número único, classe, assento, preço e validade
- **Reserva** — associação Passageiro ↔ Voo, contém Bilhete por composição
- **GestorCompanhia** — controlador central de todas as operações

Relações UML

Composição: A classe Reserva compõe a classe Bilhete. O Bilhete é criado no construtor da Reserva e só existe no seu contexto; ao cancelar a Reserva, o Bilhete é automaticamente invalidado.

Associação: A Reserva associa Passageiro e Voo numa relação de atendimento. O Voo associa a Aeronave que o opera. O GestorCompanhia agrega listas independentes de Aeronaves, Voos, Passageiros e Reservas.

ESTRUTURA DAS CLASSES

Classe Passageiro

Responsável por armazenar os dados pessoais de cada passageiro da companhia.

Atributos: Id, Nome, NumeroPassaporte, Email, Telefone, DataNascimento

Métodos: Idade(), ActualizarDados(), ExibirDados()

Classe Aeronave

Responsável por gerir a frota, incluindo a capacidade e o estado de ocupação de todos os assentos, organizados por classe.

Atributos: Matricula, Tipo, CapacidadeTotal, Assentos (List<Assento>)

Métodos: LugaresDisponiveisPorClasse(), TotalLugaresDisponiveis(), CalcularOcupacao(), ProcurarAssentoLivre(), ProcurarAssentoPorNumero()

Classe Assento

Representa um assento individual na aeronave, com o seu número identificador, a classe a que pertence e o estado de ocupação.

Atributos: Numero, Classe (ClasseVoo), Ocupado

Métodos: Ocupar(), Libertar()

Classe Voo

Representa um voo, associando uma Aeronave a uma rota e horário específicos. Gere as reservas e calcula o preço do bilhete com base na antecedência da compra.

Atributos: Id, NumeroVoo, Origem, Destino, DataHoraPartida, DataHoraChegada, Aeronave, Reservas, Activo, _precoBase (Dictionary)

Métodos: CalcularPreco(), CapacidadeDisponivel(), AdicionarReserva(), RemoverReserva(), Duracao(), ListarPassageiros(), ExibirInformacoes()

Classe Bilhete

Representa o bilhete de passagem associado a uma reserva. É criado automaticamente no momento da reserva e invalida-se aquando do cancelamento (composição com Reserva).

Atributos: Id, NumeroBilhete, Classe, NumeroAssento, Preco, DataEmissao, Valido

Métodos: EmitirBilhete(), Invalidar(), ExibirBilhete()

Classe Reserva

Responsável pelo registo de uma reserva, associando um Passageiro a um Voo numa determinada classe. Compõe um Bilhete e referencia o Assento ocupado na aeronave. Gere o ciclo de vida completo: reserva → check-in → embarque → cancelamento.

Atributos: Id, IdReserva, Passageiro, Voo, DataReserva, Estado (EstadoReserva), Bilhete, _assento

Métodos: EfectuarReserva(), CancelarReserva(), RealizarCheckIn(), ConfirmarEmbarque(), AlterarEstado(), ExibirInformacoes()

Classe GestorCompanhia

Controlador central que coordena todas as operações do sistema, mantendo as listas globais de aeronaves, voos, passageiros e reservas.

Atributos: NomeCompanhia, Aeronaves, Voos, Passageiros, Reservas (listas genéricas List<T>)

Métodos: CadastrarAeronave(), CadastrarPassageiro(), CriarVoo(), EfectuarReserva(), CancelarReserva(), RealizarCheckIn(), ConfirmarEmbarque(), ListarPassageiros(), PesquisarVoos(), EmitirRelatorio()

IMPLEMENTAÇÃO DO SISTEMA

O sistema foi desenvolvido em linguagem C# com .NET Framework 4.6, organizado nos seguintes namespaces:

- **SistemaCompanhiaAerea.Models** — classes de domínio (Passageiro, Aeronave, Assento, Voo, Bilhete, Reserva)
- **SistemaCompanhiaAerea.Enums** — ClasseVoo e EstadoReserva
- **SistemaCompanhiaAerea.Exceptions** — exceções personalizadas
- **SistemaCompanhiaAerea.Services** — GestorCompanhia
- **SistemaCompanhiaAerea** — Program.cs com menu interativo

Estrutura de Ficheiros

```
SistemaCompanhiaAerea/  
■■■ SistemaCompanhiaAerea.sln  
■■■ SistemaCompanhiaAerea/  
■■■ Program.cs Ponto de entrada + menu interativo  
■■■ Enums/  
■ ■■■ ClasseVoo.cs Economica, Executiva, PrimeiraClasse  
■ ■■■ EstadoReserva.cs Activa, Cancelada, CheckInRealizado, Embarcado  
■■■ Exceptions/  
■ ■■■ AssentoOcupadoException.cs  
■ ■■■ VooLotadoException.cs  
■ ■■■ CheckInForaDoPrazoException.cs  
■■■ Models/  
■ ■■■ Assento.cs  
■ ■■■ Aeronave.cs  
■ ■■■ Voo.cs  
■ ■■■ Passageiro.cs  
■ ■■■ Bilhete.cs  
■ ■■■ Reserva.cs  
■■■ Services/  
■■■ GestorCompanhia.cs Coordenação central
```

Menu Interativo

O Program.cs apresenta um menu principal com seis opções funcionais e sub-menus para cada área, com dados de demonstração pré-carregados ao iniciar (3 aeronaves, 4 voos, 5 passageiros e 5 reservas), permitindo testar imediatamente todas as funcionalidades do sistema.

- 1. Gestão de Aeronaves — cadastrar, listar, ver detalhes
- 2. Gestão de Voos — criar, listar, pesquisar por rota, ver detalhes, listar passageiros
- 3. Gestão de Passageiros — cadastrar, listar, pesquisar
- 4. Gestão de Reservas e Bilhetes — efectuar, cancelar, ver bilhete, listar
- 5. Check-In e Embarque — realizar check-in, confirmar embarque
- 6. Relatório Geral — aeronaves, voos, reservas por estado e receita total

CONCEITOS DE POO APLICADOS

Encapsulamento

Todos os atributos das classes foram encapsulados com propriedades públicas de leitura (**get**) e escrita privada (**private set**), garantindo a integridade dos dados. A validação dos argumentos é feita nos construtores com lançamento de **ArgumentException** ou **ArgumentNullException** em caso de dados inválidos.

Composição

A classe **Reserva** possui composição com **Bilhete**: o Bilhete é criado no construtor da Reserva (`new Bilhete(...)`) e não existe de forma independente. Ao cancelar a Reserva, o método **Bilhete.Invalidar()** é chamado automaticamente, tornando o bilhete inválido.

Associação

O **GestorCompanhia** agrega Aeronaves, Voos, Passageiros e Reservas em listas genéricas (**List<T>**), mantendo referências a objectos que existem de forma independente. A classe **Voo** associa-se a **Aeronave**; a classe **Reserva** associa-se a **Passageiro** e a **Voo**.

Enums

ClasseVoo (Económica, Executiva, PrimeiraClasse) é utilizado para classificar os assentos da aeronave, filtrar a disponibilidade por classe e calcular o preço base do bilhete. **EstadoReserva** (Activa, Cancelada, CheckInRealizado, Embarcado) controla o ciclo de vida completo da reserva, com validações de transição de estado nos métodos da classe Reserva.

Coleções Genéricas e LINQ

Foram utilizadas listas genéricas (**List<T>**) para armazenar aeronaves, assentos, voos, passageiros e reservas, e um **Dictionary<ClasseVoo, decimal>** para os preços base por classe. LINQ foi aplicado extensivamente em filtragens, ordenações e projecções, como na listagem de passageiros com reservas activas, no cálculo da receita total e na pesquisa de voos por rota.

Excepções Personalizadas

Foram implementadas três excepções: **AssentoOcupadoException** — lançada quando se tenta reservar um assento já ocupado; **VooLotadoException** — lançada quando não existem lugares disponíveis na classe pretendida; **CheckInForaDoPrazoException** — lançada quando o check-in é efectuado fora da janela permitida (entre 24h e 1h antes da partida).

FUNCIONALIDADE ESPECIAL: CÁLCULO DE PREÇO DINÂMICO

O sistema implementa um mecanismo de preço dinâmico baseado na antecedência da compra em relação à data de partida do voo. Este mecanismo é implementado no método

Voo.CalcularPreco() e é invocado automaticamente na criação de cada Reserva.

Antecedência da compra	Multiplicador	Variação
60 dias ou mais	× 0,70	30% desconto
30 a 59 dias	× 0,85	15% desconto
14 a 29 dias	× 1,00	Preço base
7 a 13 dias	× 1,15	15% extra
Menos de 7 dias	× 1,30	30% extra

Este cálculo incentiva a compra antecipada de bilhetes, favorecendo o planeamento da companhia e penalizando reservas de última hora com preços mais elevados. O preço calculado é arredondado a dois decimais e registado no bilhete no momento da emissão.

Dados de Demonstração (Pré-carregados ao Iniciar)

Ao iniciar o sistema, o método **InicializarDadosDemonstracao()** carrega automaticamente os seguintes dados de exemplo:

Categoria	Qtd.	Exemplos
Aeronaves	3	CS-TTA (Airbus A320), CS-TTB (Boeing 737-800), CS-TTC (Embraer E190)
Voos	4	AP101 LIS→OPO, AP202 LIS→FAO, AP303 OPO→LIS, AP404 LIS→PDL
Passageiros	5	Maria Silva, João Santos, Ana Ferreira, Carlos Mendes, Sofia Costa
Reservas	5	1 Executiva, 3 Económicas, 1 Primeira Classe (com check-in)

RESULTADOS OBTIDOS

O sistema desenvolvido conseguiu atingir todos os objectivos propostos:

- Cadastro e listagem de aeronaves, voos e passageiros
- Criação de voos com aeronave associada, rota e preços por classe
- Efectuação de reservas com atribuição automática de assento e cálculo de preço dinâmico
- Emissão gráfica de bilhetes de passagem com todos os dados do voo e passageiro
- Cancelamento de reservas com libertação automática do assento e invalidação do bilhete
- Realização de check-in com validação da janela permitida (24h a 1h antes da partida)
- Confirmação de embarque após check-in realizado
- Tratamento correcto de todas as excepções personalizadas
- Relatório geral com contagem de reservas por estado e receita total calculada
- Menu interactivo funcional com dados de demonstração pré-carregados

O sistema apresentou funcionamento correcto durante os testes realizados, compilando sem erros em C# 6 com .NET Framework 4.6, no ambiente Visual Studio 2015.

CONCLUSÃO

Conclui-se que o desenvolvimento do Sistema de Gestão de Companhia Aérea permitiu aplicar de forma prática e consistente os conceitos de Programação Orientada a Objectos em C#.

O projecto demonstrou a importância da modelação UML, do encapsulamento, da composição, da associação, dos enums, das colecções genéricas, do LINQ e do tratamento de excepções no desenvolvimento de sistemas robustos, organizados e de fácil manutenção.

A implementação do preço dinâmico por antecendência e do controlo de check-in com janela temporal revelaram-se fundamentais para garantir a integridade operacional do sistema, simulando comportamentos reais de uma companhia aérea.

A relação de composição entre Reserva e Bilhete evidenciou a correcta aplicação dos princípios de POO, assegurando que o ciclo de vida do bilhete está sempre vinculado ao da reserva correspondente.

Os objectivos definidos inicialmente foram alcançados com sucesso, resultando num sistema funcional, organizado e totalmente compatível com os requisitos propostos pelo docente orientador.